Using the Class Database API in Lessons

Teacher guide · the ishflask Flask + SQLite tool · Theme A — Databases

A small Flask + SQLite web API. Open a link (or run a few lines of Python) and the server sends back data from a database. It makes the **client–server** database idea real, and doubles as a starting template for student IAs. Repo: github.com/tanasel/ishflask.

Two ways to use it

- A live demo for the Databases unit (queries over HTTP, client and server).
- A copyable template for students whose IA is a database-backed app.

Part 1 — A live demo in the lesson

Once it is deployed at `api.ishweb.nl` (or run locally at `127.0.0.1:5000`), project these and click / visit:

- The landing page — click **Load all patients** and a table fills in. Browser asks the server → server runs SQL → returns JSON.
- `/patients` — the raw JSON the server returns.
- `/appointments/D1` — a live JOIN (one doctor's patients).
- `/patients/search?name=Anika` — a parameterised query.
- Hand students `client_example.py` (a 3-line `requests.get`) — they pull data into their own Python without ever holding the .db file. This is exactly "Option 2 — Python on the server."

"Drop a .db, get an API" for class practice

Put any practice `.db` into the server's `databases/` folder and it is instantly queryable at `/db/<name>/<table>` — no extra code. A student's practice database becomes a live read-only API in seconds.

Run it locally (no hosting needed)

```
git clone https://github.com/tanasel/ishflask.git
cd ishflask
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python seed_medical.py
flask --app app run        # http://127.0.0.1:5000
```

Part 2 — Students using it for their IA

Hand students the student handout. In short: they design their own .db, clone the template, write their own routes, add read + write (INSERT/UPDATE/DELETE), and run it locally for their video.

Marking integrity — what must be the student's own

The template is a skeleton only. For the work to count, the student must provide:

- their own database design,
- their own routes,
- their own SQL, and
- read + write logic (the medical example removed).

A student who only swaps the .db and changes nothing else has not demonstrated the programming.

> **Honest scope note:** Not every database IA needs a web app. A plain local Python + sqlite3 program is a perfectly valid IA and is simpler for many students. Use this template for students who want a web / API-style project.

The template is read-only by design

Every connection opens the database read-only (safe for a shared class demo). For their IA, students add write routes (a normal connection without read-only mode). That is a feature: it forces the INSERT / UPDATE / DELETE that A3.3.3 expects.

Deploying the shared class API — status

- Built, tested, and on GitHub (tanasel/ishflask).
- Going live at api.ishweb.nl is currently blocked by an ICDSoft hosting issue (a false "not enough disk space" — the account has ~24 GB free). A support ticket resolves it; once cleared, deployment is about 5 minutes (steps in the repo README).
- Until then, run it locally for demos — everything behaves the same.

Files at a glance

- `app.py` — the API (read-only; students add write routes)
- `seed_medical.py` — builds the sample database
- `static/index.html` — the landing page
- `client_example.py` — the Python client students copy
- `README.md` — run + deploy guide
- `docs/teaching-additions.md` — paste-ready additions for your SQLite doc + slides